ADITYA UNIVERSITY

SCHOOL OF ENGINEERING

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Cornerstone project work entitled "**STUDENT ATTENDANCE MONITORING SYSTEM**" is submitted by

K. Sai Rahul Raj	**(25B21CS043)**
M. Durga Prasad	**(25B21CS079)**
M. Akshaya	**(24B11CS269)**
K. Anusha	**(24B11CS200)**

in partial fulfillment of the requirements for the award of the B.Tech. degree in COMPUTER SCIENCE AND ENGINEERING for the academic year 2025-2026.

Project Guide
Mrs. R.Padma Sri,
Assistant Professor,
Department of CSE

Head of the Department
Dr. T. Sudha Rani,
Assistant Professor,
Department of CSE

DECLARATION

We hereby declare that the Cornerstone project entitled **"STUDENT ATTENDENCE MONITORING SYSTEM** is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. degree** in **Computer Science and Engineering**. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy

Place: Surampalem

Date:

By

K. Sai Rahul Raj	(25B21CS043)
M. Durga Prasad	(25B21CS079)
M. Akshaya	(24B11CS269)
K. Anusha	(24B11CS200)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our **STUDENT ATTENDENCE MONITORING SYSTEM 2.0 Project Guide, R. Padma Sri, Assistant Professor,** who has guided us throughout the project. His valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department,**Dr. K. Chandra Sekhar, Associate Professor** and **Dr. Hari Jyothula, Associate Professor,** Department of CSE, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. T Sudha Rani, Associate Professor**, for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Engineering** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor - Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Student Attendence Monitoring System project successfully.

Table of contents

CONTENTS	PAGE NO
ABSTRACT	i
LIST OF FIGURES	ii
LIST OF TABLES	iii
ACRONYMS	iv
1. INTRODUCTION	1
1.1 Brief Information about the Project	1
1.2 Motivation and Contribution of Project	1
1.3 Objectives of the Project	1
1.4 Scope of the Project	2
2. SYSTEM ANALYSIS	3
2.1 Functional Requirements	3
2.2 Non-Functional Requirements	4
2.3 Requirements Specification	4
2.3.1 Minimum Hardware Requirements	4
2.3.2 Software Requirements	4
3. TECHNOLOGY DESCRIPTION	5
3.1 Programming Language	5
3.1.1 Introduction to JavaScript	5
3.2 Framework / Libraries	5
3.2.1 Introduction to React	5
3.2.2 Introduction to Node.js	5
3.2.3 Introduction to Tailwind CSS	6
3.3 Database Technology	6
3.3.1 MongoDB	6
3.4 Tools & IDEs	6
3.4.1 Visual Studio Code	
4. DATABASE DESIGN	7
4.1 Database Tables	7
4.2 Table Structure	7
4.3 Keys and Constraints	7
4.4 Table Relationships	8
5. SYSTEM DESIGN	9
5.1 System Architecture	9
5.2 Modules Description	10
6. SYSTEM IMPLEMENTATION	11

7. CONCLUSION 15
8. FUTURE ENHANCEMENTS 16
9. REFERENCES 17
APPENDIX (Sample Code) 18
APPENDIX (MongoDB Queries) 20

ABSTRACT

The STUDENT ATTENDANCE MONITORING SYSTEM is a web-based application designed to efficiently track, manage, and report student attendance across academic programs. The system replaces traditional paper-based or spreadsheet-driven attendance methods with a modern, real-time digital platform that enables faculty to record attendance, administrators to generate reports, and students to monitor their own attendance status.

The application provides role-based access control to distinguish between administrators, faculty, and students. Faculty can mark daily attendance, view attendance summaries by subject, and generate class-wise reports. Administrators can manage users, departments, subjects, and export comprehensive attendance data. Students can log in and track their attendance percentage in real time.

Developed using the MERN-inspired stack with React for the frontend, Node.js with Express.js for the backend REST API, and MongoDB for structured relational data storage, the system ensures high performance, data integrity, and a responsive user interface built with Tailwind CSS. With its intuitive interface and efficient data handling capabilities, this provides a scalable and reliable solution for attendance management in educational institutions.

LIST OF FIGURES

Figure no.	Figure Title	Page no.
1	Login Screen of Attendance Management System Dashboard	10
2	Admin Dashboard Screen of Attendance Management System Dashboard	10
3	Mark Attendance Screen of Attendance Management System Dashboard	10
4	Attendance Report Screen of Attendance Management System Dashboard	10
5	Student View Screen of Attendance Management System Dashboard	11
6	User Management Screen of Attendance Management System Dashboard	11

LIST OF TABLES

Table no.	Table Title	Page no.
1	Students Table Structure	7
2	Faculty Table Structure	7
3	Subjects Table Structure	8
4	Attendance Table Structure	8

ACRONYMS

- **AMSD** : Attendance Management System Dashboard
- **API** : Application Programming Interface
- **UI** : User Interface
- **UX** : User Experience
- **HTTP** : HyperText Transfer Protocol
- **HTTPS** : HyperText Transfer Protocol Secure
- **JSON** : JavaScript Object Notation
- **JWT** : JSON Web Token
- **SQL** : Structured Query Language
- **ORM** : Object Relational Mapping
- **REST** : Representational State Transfer
- **CRUD** : Create, Read, Update, Delete
- **CSS** : Cascading Style Sheets
- **DOM** : Document Object Model

1. INTRODUCTION

1.1 Brief Information about the Project

The STUDENT ATTENDANCE MONITORING SYSTEM is a web-based application developed to automate and streamline the process of recording and monitoring student attendance in educational institutions. It provides a centralized platform for faculty to mark attendance per subject and class, and for administrators to manage academic data including departments, subjects, faculty, and students. The system replaces paper-based attendance registers with an efficient digital solution that reduces errors, saves time, and provides instant access to attendance analytics.

The system is built using modern web technologies including React with Tailwind CSS for the responsive frontend dashboard, Node.js and Express.js for the backend REST API server, and MongoDB as the relational database for structured data storage. It features secure role-based authentication using JWT tokens, real-time attendance dashboards, subject-wise attendance tracking, and report generation.

1.2 Motivation and Contribution of Project

Traditional attendance management in colleges and schools relies on paper-based registers or basic spreadsheets, which introduce inefficiencies such as manual data entry errors, difficulty aggregating attendance across subjects, time-consuming report generation, and lack of student visibility into their own attendance status.

The key contributions of this project include:

- Automated digital attendance recording with instant database persistence
- Role-based access for administrators, faculty, and students
- Real-time attendance dashboards with subject-wise and student-wise summaries
- Automated attendance percentage calculation with shortage alerts
- Report generation and export functionality for academic records
- Secure authentication with JWT-based session management

This system simplifies attendance management and significantly improves operational efficiency for educational institutions.

1.3 Objectives of the Project

The main objectives of the Attendance Management System Dashboard are:

- To develop a responsive web application for managing student attendance

To store attendance data in a structured relational database

- To provide role-based access to administrators, faculty, and students
- To enable faculty to mark and update attendance per subject and session
- To display real-time attendance summaries and analytics on dashboards
- To calculate attendance percentage and flag students with attendance shortage
- To generate and export attendance reports in structured formats

1.4 Scope of the Project

The Attendance Management System Dashboard can be deployed in colleges, universities, schools, and training institutes to manage attendance efficiently. It supports operations such as user registration and role assignment, student and subject management, day-wise attendance marking, view of attendance history per subject, and automated percentage computation. The system provides a clean and responsive interface accessible on desktops and tablets, enabling institutions to maintain accurate attendance records in a centralized digital platform.

2. SYSTEM ANALYSIS

2.1 Functional Requirements

Functional requirements define the core operations that the Attendance Management System Dashboard must perform:

User Authentication and Role Management

The system should allow users to log in securely with credentials and assign roles (Admin, Faculty, Student). Each role grants access to specific features and data. JWT tokens should manage session security.

Student Management

The system should allow administrators to add, update, view, and delete student records including name, roll number, email, department, and enrolled subjects.

Faculty Management

Administrators should be able to manage faculty profiles and assign them to specific subjects and departments.

Subject and Department Management

The system should support creation and management of departments and subjects. Subjects should be linked to departments and assigned faculty members.

Mark Attendance

Faculty should be able to mark attendance for students enrolled in their assigned subjects on a session-by-session basis. Each attendance record should include student ID, subject ID, date, and status (Present/Absent).

View and Edit Attendance

Faculty and administrators should be able to view, modify, and correct attendance records. Previous sessions should remain accessible for corrections.

Attendance Dashboard and Reports

The system should provide dashboards showing attendance summaries, subject-wise attendance percentages per student, and class-wide trends. Reports should be exportable.

Student Self-View

Students should be able to log in and view their own attendance record per subject, including present/absent counts and attendance percentage.

2.2 Non-Functional Requirements

Non-functional requirements define quality attributes the system must satisfy:

- Performance: The system should load dashboards and process attendance submissions within 2 seconds under normal load
- Security: Role-based access control must prevent unauthorized data access; all endpoints must be protected with JWT authentication
- Usability: The interface should be intuitive and clean, allowing faculty to mark attendance within 3 clicks
- Reliability: The system should handle concurrent users and maintain data consistency without record loss
- Scalability: The architecture should support hundreds of students and multiple departments without performance degradation
- Availability: The system should be available 24/7 with minimal downtime.

2.3 Requirements Specification

This section specifies the hardware and software needed to run the system.

2.3.1 Minimum Hardware Requirements

- Processor: Intel i5 or above
- RAM: 8 GB (minimum)
- Hard Disk: 50 GB free space
- Network: Stable internet connection

2.3.2 Software Requirements

- Operating System: Windows 10/11, macOS, or Linux
- Programming Language: JavaScript (ES6+)
- Frontend Framework: React with Tailwind CSS
- Backend Framework: Node.js with Express.js
- Database: MySQL
- IDE: Visual Studio Code
- Browser: Chrome, Firefox, Safari, or Edge (latest versions)

3. TECHNOLOGY DESCRIPTION

3.1 Programming Language

3.1.1 Introduction to JavaScript

JavaScript is a high-level, dynamic, and versatile programming language that has become the foundation of modern web development. It enables interactive frontend experiences in web browsers and, through Node.js, also powers server-side backend services. In the Attendance Management System Dashboard, JavaScript is used across the full stack — React on the frontend and Node.js with Express on the backend — allowing a unified development language for the entire application. Its asynchronous capabilities via Promises and async/await are critical for handling database queries and API calls efficiently.

3.2 Framework / Libraries

3.2.1 Introduction to React

React is an open-source JavaScript library developed by Meta (Facebook) for building fast and interactive user interfaces. It follows a component-based architecture where the UI is broken into reusable, self-contained components that manage their own state. In this project, React powers the entire frontend dashboard including login screens, attendance marking forms, student dashboards, and administrative panels. React's Virtual DOM ensures efficient re-rendering, making attendance dashboards update smoothly in response to data changes.

3.2.2 Introduction to Node.js

Node.js is a JavaScript runtime environment built on Chrome's V8 JavaScript engine that enables JavaScript to be executed on the server side. Its event-driven, non-blocking I/O model makes it highly efficient for handling concurrent API requests, which is essential in a multi-user attendance system. In this project, Node.js powers the backend server responsible for processing API requests, managing authentication, executing database queries, and serving responses to the React frontend.

3.2.3 Introduction to Tailwind CSS

Tailwind CSS is a utility-first CSS framework that allows developers to build responsive and well-designed user interfaces directly within HTML markup using predefined utility classes. Unlike traditional CSS frameworks, Tailwind does not impose a specific design or component set, giving developers complete control over the visual output. In this project, Tailwind CSS is used to style all frontend components of the AMSD, ensuring

a consistent, clean, and responsive design across desktops and tablets without writing custom CSS files.

3.3 Database Technology

The Attendance Management System Dashboard uses MongoDB, a widely adopted open-source relational database management system, for storing all structured application data. MongoDB uses for defining, querying, and manipulating data. Its support for foreign keys and relational integrity makes it ideal for modeling the relationships between students, faculty, subjects, departments, and attendance records that are central to this application.

3.3.1 MongoDB

MongoDB is a robust, open-source relational database that organizes data into structured tables with defined schemas and relationships enforced through constraints. In the Attendance Management System Dashboard, MongoDB stores four primary tables: Students, Faculty, Subjects, and Attendance. It supports complex JOIN queries needed to generate attendance reports and summary dashboards, ensures data integrity through primary and foreign key constraints, and provides indexing for fast lookups on student IDs and dates.

3.4 Tools & IDEs

3.4.1 Visual Studio Code

Visual Studio Code (VS Code) is used as the primary development environment for the Attendance Management System Dashboard. It is a lightweight yet powerful source code editor that supports JavaScript, React, and Node.js development through a rich ecosystem of extensions. VS Code provides features such as IntelliSense code completion, integrated terminal, Git version control integration, live debugging support, and extensions like ESLint and Prettier for code quality, which significantly improve development productivity.

4. DATABASE DESIGN

4.1 Database Tables

The Attendance Management System Dashboard uses a MongoDB relational database to store all application data in a structured and consistent manner. The database consists of four primary tables: Students, Faculty, Subjects, and Attendance.

- The Students table stores student account information, credentials, and department association
- The Faculty table stores faculty profile and login credentials
- The Subjects table stores subject information and links subjects to faculty and departments
- The Attendance table stores individual attendance records per student per subject per date

4.2 Table Structure

4.2.1 Students Table

Field Name	Data Type	Description	Constraint
student_id	OBJECTid	Unique Student ID	Primary Key
roll_number	OBJECTid	Student Roll Number	Unique, Not Null
name	STRING	Student Full Name	Not Null
email	STRING	Student Email	Unique, Not Null
department_id	OBJECTID	Department Reference	Foreign Key
password_hash	STRING	Hashed Password	Not Null

Table 1: Students Table Structure

4.2.2 Faculty Table

Field Name	Data Type	Description	Constraint
faculty_id	OBJECTID	Unique Faculty ID	Primary Key
name	STRING	Faculty Full Name	Not Null

Field Name	Data Type	Description	Constraint
email	STRING	Faculty Email	Unique, Not Null
department_id	OBJECTID	Department Reference	Foreign Key
password_hash	STRING	Hashed Password	Not Null

Table 2: Faculty Table Structure

4.3 Keys and Constraints

Keys and constraints maintain data integrity and ensure accurate data storage:

- Primary Keys: attendance_id, student_id, faculty_id, subject_id in their respective tables
- Foreign Keys: student_id and subject_id in Attendance table; faculty_id and department_id in Subjects table
- Unique Constraints: roll_number, email across Students and Faculty tables; subject_code in Subjects
- NOT NULL Constraints: All critical fields including names, emails, dates, and status cannot be empty
- ENUM Constraint: Attendance status is restricted to 'P' (Present) or 'A' (Absent)
- Indexing: Fields used in frequent queries (student_id, subject_id, date) are indexed for performance

These constraints help in maintaining consistency and avoiding duplicate or invalid data.

4.4 Table Relationships

The database tables are related through foreign key references that model real-world academic relationships:

- Each Attendance record references exactly one Student and one Subject
- Each Subject is assigned to one Faculty member and belongs to one Department
- Each Student belongs to one Department

5. SYSTEM DESIGN

5.1 System Architecture

The Attendance Management System Dashboard follows a three-layer client-server architecture:

1. Presentation Layer (User Interface)

- Developed using React with Tailwind CSS
- Provides role-specific dashboards for Admin, Faculty, and Student
- Handles form submissions, API calls, and real-time dashboard rendering
- Communicates with backend via RESTful HTTP API calls

1. Application Layer (Business Logic)

- Built with Node.js and Express.js
- Processes authentication and authorization using JWT tokens
- Validates incoming request data before database operations
- Executes business rules such as attendance percentage computation and shortage detection
- Routes requests to appropriate database queries and returns JSON responses

1. Data Layer (Database)

- Managed using MySQL relational database
- Stores all entities: Students, Faculty, Subjects, Departments, Attendance records
- Enforces data integrity through constraints and foreign key relationships
- Supports complex JOIN queries for dashboard reports and attendance summaries

5.2 Modules Description

The system is organized into the following functional modules:

1. Authentication Module

- Handles user login for all three roles: Admin, Faculty, Student
- Validates credentials against hashed passwords stored in the database
- Issues JWT tokens upon successful authentication

- Protects all API endpoints using token-based middleware

2. Admin Module

- Manages student and faculty records (Create, Read, Update, Delete)
- Manages departments and subjects
- Assigns faculty to subjects
- Views system-wide attendance reports and exports data

3. Attendance Marking Module

- Allows faculty to select a subject and session date
- Displays enrolled students for the selected subject
- Enables marking each student as Present or Absent
- Persists records to the Attendance table with timestamp

4. Attendance Report Module

- Computes attendance percentage per student per subject
- Flags students with attendance below the minimum threshold (e.g., 75%)
- Generates subject-wise and student-wise attendance summaries
- Provides export functionality for academic reporting

5. Student Dashboard Module

- Allows students to log in and view their own attendance records
- Displays subject-wise present/absent counts and attendance percentage
- Shows visual progress indicators for attendance status

6. Database Module

- Connects to MySQL using a connection pool for efficient query handling
- Executes CRUD operations for all entities
- Manages transactions for multi-step operations

6. SYSTEM IMPLEMENTATION

Login Screen:

Displays the login interface where users enter their registered email and password. The system authenticates credentials and redirects users to their role-specific dashboard — Admin, Faculty, or Student panel.

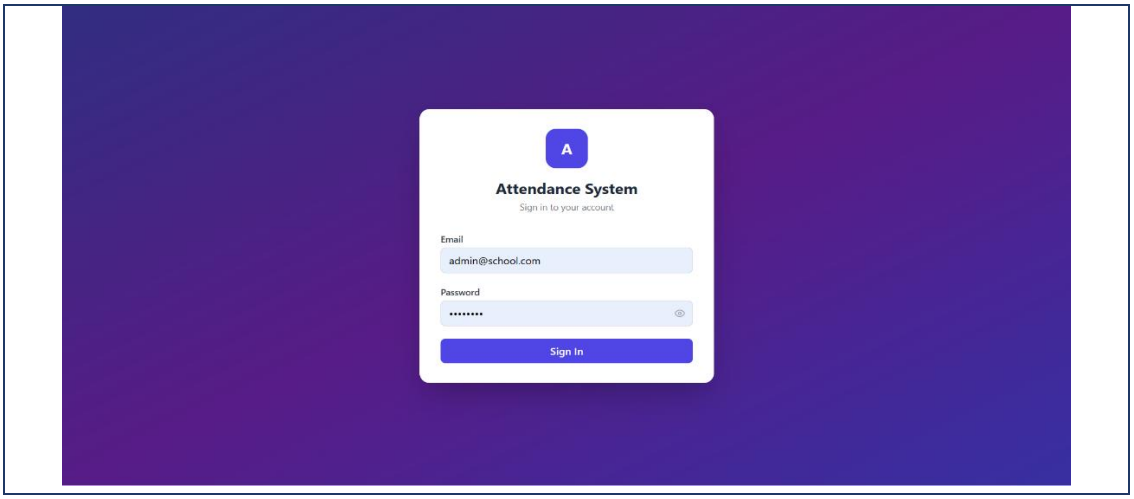

Figure 1: Login Screen of Attendance Management System Dashboard

Admin Dashboard

Provides the administrator with a centralized overview panel showing total registered students, faculty, subjects, and system-wide attendance statistics. Quick navigation links allow access to all management modules.

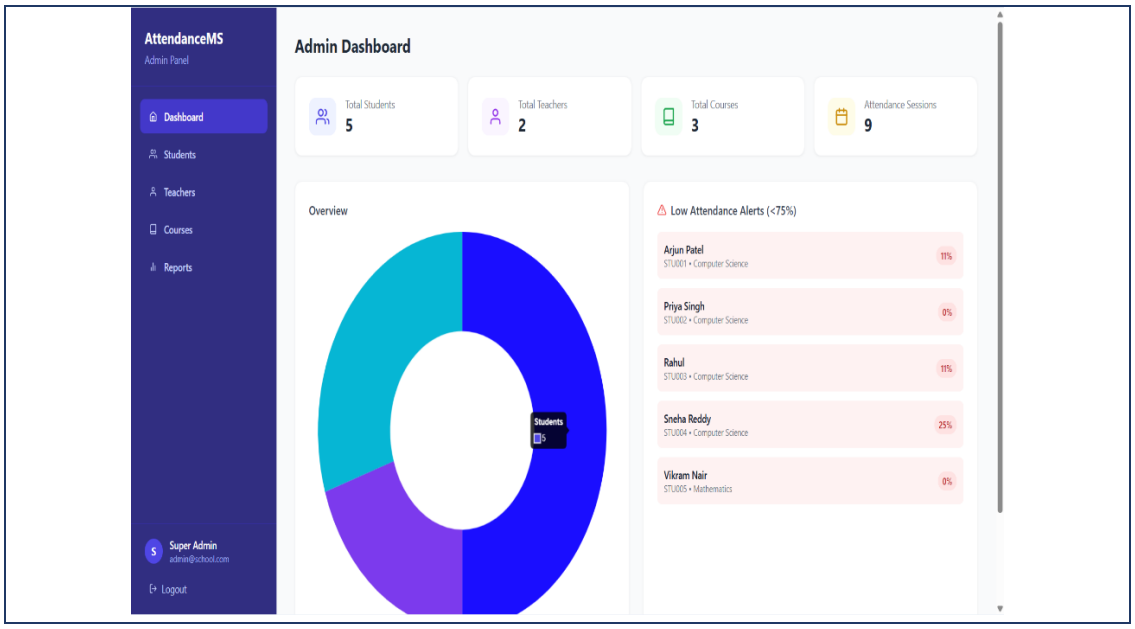

Figure 2: Admin Dashboard Screen of Attendance Management System Dashboard

Mark Attendance Screen:

Enables faculty to select a subject from their assigned subjects, choose the session date, and mark each enrolled student as Present or Absent. The form validates that attendance is not marked twice for the same session and saves records to the database instantly.

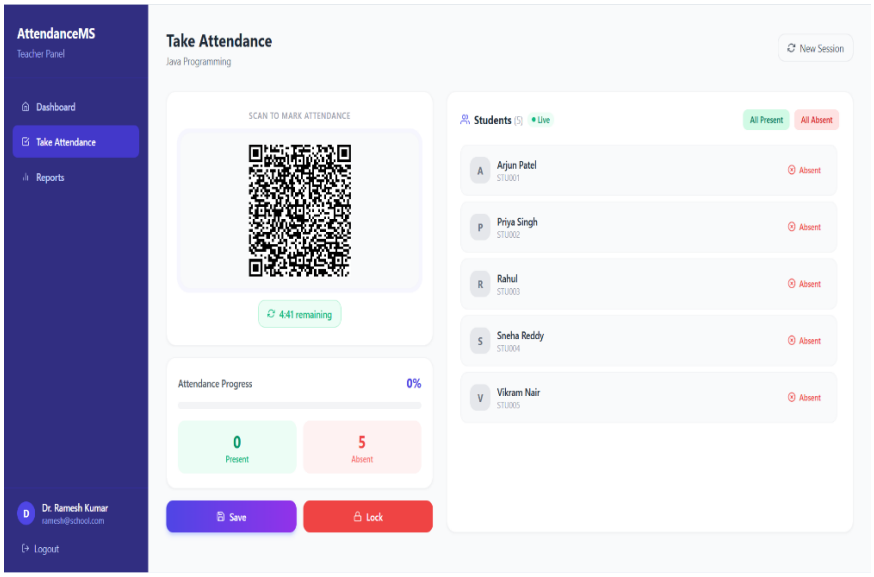

Figure 3: Mark Attendance Screen of Attendance Management System Dashboard

Attendance Report Screen:

Displays a comprehensive report showing each student's attendance percentage per subject. Students with attendance below the required threshold are highlighted with a shortage indicator. The report can be filtered by subject, department, or date range.

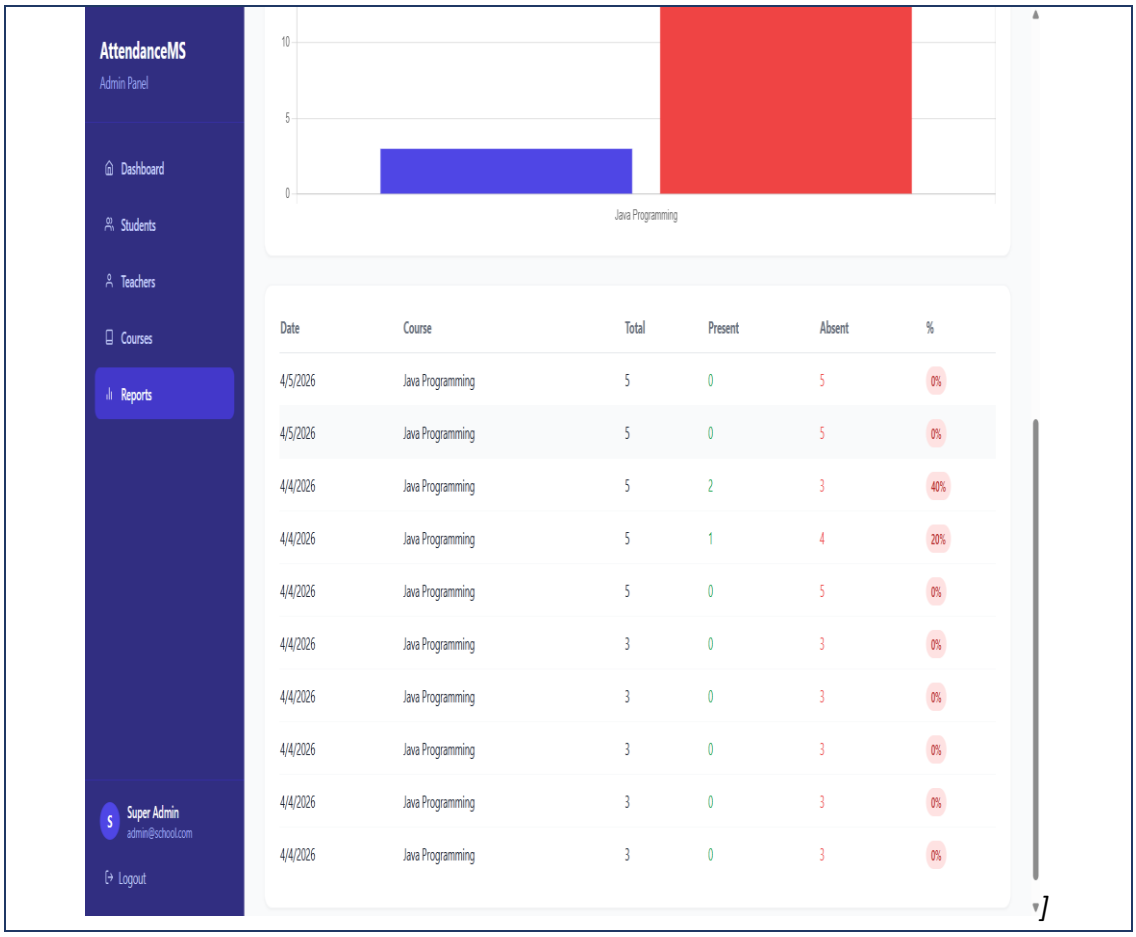

Figure 4: Attendance Report Screen of Attendance Management System Dashboard

Student Self-View Screen:

Allows students to log in and view their own subject-wise attendance summary. Each subject displays the number of classes held, classes attended, classes absent, and the computed attendance percentage with a visual progress indicator.

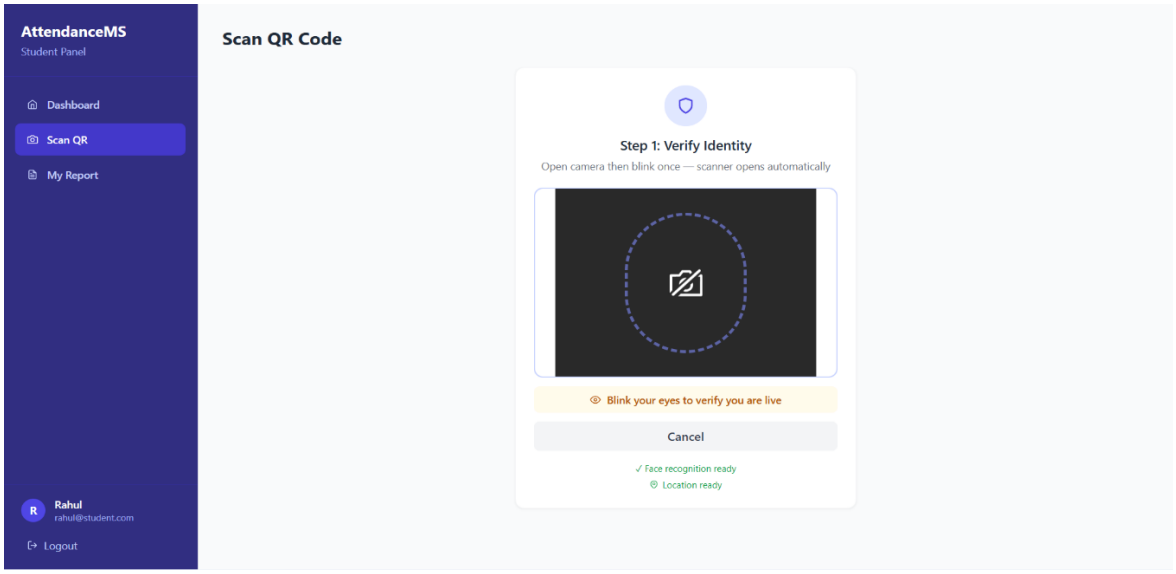

Figure 5: Student View Screen of Attendance Management System Dashboard

User Management Screen:

Enables administrators to add, edit, and delete student and faculty records. The screen displays a searchable data table with all registered users and provides forms for adding new users or updating existing records.

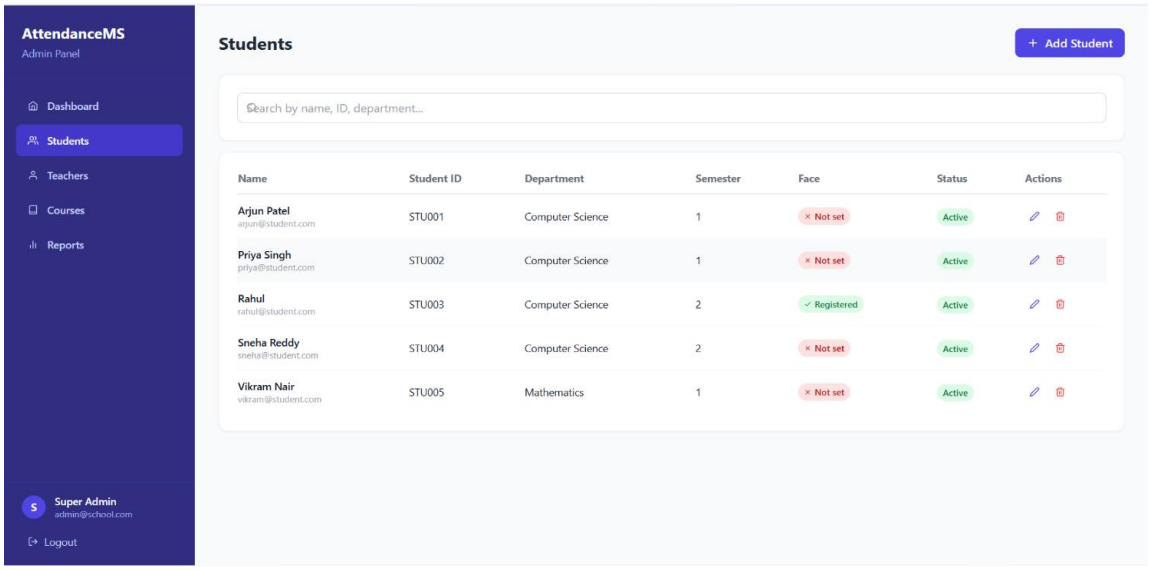

Figure 6: User Management Screen of Attendance Management System Dashboard

7. CONCLUSION

The Attendance Management System Dashboard has been successfully developed to provide educational institutions with a modern, digital solution for tracking and managing student attendance. The system replaces manual attendance registers with an efficient web-based platform, significantly reducing administrative effort, eliminating data entry errors, and providing instant access to attendance analytics.

The application successfully delivers its core functionalities including secure role-based authentication, subject-wise attendance marking by faculty, automated attendance percentage computation, shortage detection, and comprehensive reporting for administrators. Students benefit from real-time visibility into their own attendance status across all enrolled subjects.

By leveraging the power of React for a responsive frontend, Node.js with Express.js for a scalable API backend, and MySQL for structured relational data management, the system ensures reliable performance, data integrity, and extensibility. The modular architecture ensures that each component of the system is independently maintainable and testable.

Overall, the Attendance Management System Dashboard meets all defined project objectives and provides a scalable foundation that can be enhanced with additional features such as biometric integration, mobile applications, automated email notifications for attendance shortage, and advanced analytics dashboards.

8. FUTURE ENHANCEMENTS

The Attendance Management System Dashboard developed in this project provides essential core functionalities for attendance management. However, the system can be significantly enhanced to improve its capabilities and institutional value.

Some possible future enhancements include:

- Biometric Integration: Integrating fingerprint or facial recognition systems to enable automated, tamper-proof attendance recording without manual faculty input

- Mobile Application: Developing native iOS and Android applications to allow faculty to mark attendance and students to view their records directly from smartphones

- Automated Email/SMS Notifications: Sending automated alerts to students and parents when a student's attendance falls below the required threshold

- Advanced Analytics Dashboard: Implementing data visualization with charts and graphs showing attendance trends over time, subject-wise comparisons, and department-level summaries

- Timetable Integration: Connecting the attendance system with the academic timetable to automatically trigger attendance sessions based on the scheduled class

- Leave Management: Adding a leave application and approval workflow so that authorized absences are properly recorded and excluded from shortage calculations

- Bulk Import/Export: Supporting CSV or Excel import of student lists and export of attendance reports for integration with institutional management systems

These enhancements can make the system more autonomous, comprehensive, and suitable for large-scale institutional deployment.

References

The following references were used during the development of the Attendance Management System Dashboard:

1. React Documentation – https://reactjs.org
2. Node.js Documentation – https://nodejs.org
3. Express.js Documentation – https://expressjs.com
4. MySQL Documentation – https://dev.mysql.com/doc
5. Tailwind CSS Documentation – https://tailwindcss.com
6. JSON Web Token (JWT) – https://jwt.io
7. JavaScript: The Definitive Guide, David Flanagan, O'Reilly Media
8. MERN Stack Tutorial – GeeksforGeeks, https://www.geeksforgeeks.org/mern-stack/

Appendix

A.1 Sample Code

Database Connection Code:

The following code establishes a connection between the Node.js server and the MySQL database using the mysql2 package.

db.js:

```
const mongoose = require('mongoose');

const attendanceRecordSchema = new mongoose.Schema({
  student: { type: mongoose.Schema.Types.ObjectId, ref: 'Student',
required: true },
  status: { type: String, enum: ['present', 'absent', 'late'], default:
'absent' },
  markedAt: { type: Date },
  method: { type: String, enum: ['manual', 'qr', 'face'], default: 'manual'
},
  ip: { type: String, default: null },
  // Geo-fencing: student's location when marking attendance
  location: {
    latitude: { type: Number },
    longitude: { type: Number },
  },
});
```

Mark Attendance API Code:

This code handles the POST request to mark attendance for a list of students in a given subject and session.

attendanceController.js:

```
const attendanceSchema = new mongoose.Schema(
  {
    course: { type: mongoose.Schema.Types.ObjectId, ref: 'Course',
required: true },
    teacher: { type: mongoose.Schema.Types.ObjectId, ref: 'Teacher',
required: true },
    date: { type: Date, required: true },
    // QR session token (expires after set time)
    qrToken: { type: String },
    qrExpiresAt: { type: Date },
    // Geo-fence center (teacher's location)
    geoFence: {
      latitude: { type: Number },
      longitude: { type: Number },
      radius: { type: Number, default: 100 }, // meters
    },
    records: [attendanceRecordSchema],
    isLocked: { type: Boolean, default: false },
  },
  { timestamps: true }
);
```

```
module.exports = mongoose.model('Attendance', attendanceSchema);
```

A.2 Database Queries

Create Students Table:

```
const studentData = [
    { name: 'Arjun Patel',   email: 'arjun@student.com',   studentId:
'STU001', department: 'Computer Science', semester: 1, section: 'A', phone:
'9000000001', address: 'Hyderabad', courses: [0, 2] },
    { name: 'Priya Singh',   email: 'priya@student.com',   studentId:
'STU002', department: 'Computer Science', semester: 1, section: 'A', phone:
'9000000002', address: 'Bangalore', courses: [0, 2] },
    { name: 'Rahul Verma',   email: 'rahul@student.com',   studentId:
'STU003', department: 'Computer Science', semester: 2, section: 'A', phone:
'9000000003', address: 'Mumbai',    courses: [1] },
    { name: 'Sneha Reddy',   email: 'sneha@student.com',   studentId:
'STU004', department: 'Computer Science', semester: 2, section: 'A', phone:
'9000000004', address: 'Chennai',   courses: [1] },
    { name: 'Vikram Nair',   email: 'vikram@student.com',  studentId:
'STU005', department: 'Mathematics',      semester: 1, section: 'A', phone:
'9000000005', address: 'Pune',      courses: [2] },
  ];

  for (const s of studentData) {
    let u = await User.findOne({ email: s.email });
    if (!u) {
      u = await User.create({ name: s.name, email: s.email, password:
'student123', role: 'student' });
    }
    let student = await Student.findOne({ user: u._id });
    if (!student) {
      student = await Student.create({
        user: u._id,
        studentId: s.studentId,
        department: s.department,
        semester: s.semester,
        section: s.section,
        phone: s.phone,
        address: s.address,
        enrolledCourses: s.courses.map((i) => courses[i]._id),
      });
      console.log(`✓ Student created: ${s.email} / student123`);
    } else {
      // Update enrolledCourses if missing
      await Student.findByIdAndUpdate(student._id, {
        enrolledCourses: s.courses.map((i) => courses[i]._id),
        department: s.department,
        section: s.section,
      });
      console.log(`  Student already exists (updated): ${s.email}`);
    }
  }
```

Create Attendance Table:

```
const attendanceSchema = new mongoose.Schema(
  {
```

```
    course: { type: mongoose.Schema.Types.ObjectId, ref: 'Course',
required: true },
    teacher: { type: mongoose.Schema.Types.ObjectId, ref: 'Teacher',
required: true },
    date: { type: Date, required: true },
    // QR session token (expires after set time)
    qrToken: { type: String },
    qrExpiresAt: { type: Date },
    // Geo-fence center (teacher's location)
    geoFence: {
      latitude: { type: Number },
      longitude: { type: Number },
      radius: { type: Number, default: 100 }, // meters
    },
    records: [attendanceRecordSchema],
    isLocked: { type: Boolean, default: false },
  },
  { timestamps: true }
);

module.exports = mongoose.model('Attendance', attendanceSchema);
```

API Endpoints:

User Authentication Endpoints:

- POST /api/auth/login – User login for all roles
- GET /api/auth/logout – Session termination

Attendance Endpoints:

- POST /api/attendance/mark – Mark attendance for a session
- GET /api/attendance/:subject_id – Get attendance records by subject
- GET /api/attendance/student/:id – Get student's attendance summary
- PUT /api/attendance/:id – Update an attendance record

User Management Endpoints:

- GET /api/students – Get all students
- POST /api/students – Add new student
- PUT /api/students/:id – Update student record
- DELETE /api/students/:id – Delete student
- GET /api/subjects – Get all subjects
- POST /api/subjects – Add new subject